AI AGENT AS A FIRST-CLASS COLLABORATOR IN THE OPTIMIZATION PLATFORM

Bachelor Thesis

by

Aftab Khan

Advisor:
Dr. Andreas Karrenbauer

April 30, 2026

Contents

I. Introduction **1**
1. Context . . . 1
2. Challenges . . . 2
3. Goal . . . 3
4. Solution Outline . . . 4

II. Related Work **7**
1. LLMs and Mathematical Modeling . . . 7
2. AI Code Generation . . . 8
3. Human-in-the-loop Architecture . . . 9

III. Concept **13**
1. Overview . . . 13
2. Optimization Modeling . . . 13
3. Large Language Models (LLMs) . . . 14
4. Web Automation . . . 16
5. Chat-User Interaction . . . 17
6. Human-in-the-Loop . . . 19
7. Backend Services . . . 20

IV. Implementation **23**
1. High-Level System Architecture . . . 23
2. Solver-Panel Frontend . . . 24
3. Browser Automation Layer . . . 25
4. LLM Integration . . . 25

V. Conclusion **27**
1. Summary . . . 27
2. Current Status and Next Steps . . . 27

Bibliography **29**

A. An Appendix **31**

Sworn declaration

I declare under oath that I have prepared the paper at hand independently and without the help of others and that I have not used any other sources and resources than the ones stated. Parts that have been taken literally or correspondingly from published or unpublished texts or other sources have been labelled as such. This paper has not been presented to any examination board in the same or similar form before.

I further declare that, where AI-based tools were used during the preparation of this thesis (e.g., for language refinement, brainstorming, or code assistance), their use was critically reviewed and transparently documented where required. The academic content, argumentation, and all final decisions remain my own responsibility.

Saarbrücken, _____ _____

Chapter I.

Introduction

1. Context

1.1. Mixed-Integer Linear Programming

Linear Programming (LP) [17] is a mathematical optimization framework in which a linear objective function is optimized under a set of linear equality and inequality constraints. From a methodological perspective, LP provides a rigorous way to represent decision problems in a formal, machine-solvable structure. Owing to this property, LP has become a foundational technique in operations research, applied mathematics, and computer science, with established use in practical domains such as production planning, logistics, energy management, and economic decision support [15].

In many real-world scenarios, however, decision variables are not purely continuous. Integer Linear Programming (ILP) [11] extends LP by constraining selected variables to integer domains, thereby enabling the representation of discrete alternatives, logical dependencies, and combinatorial decisions. Mixed-Integer Linear Programming (MILP) combines continuous and integer variables in a single formulation and is therefore particularly suitable for complex planning and scheduling settings in which continuous quantities (e.g., production volumes) and discrete decisions (e.g., on/off choices, assignment decisions, routing selections) must be optimized jointly. This expressive power explains why MILP is frequently used in both industrial and research contexts despite its increased modeling and computational complexity.

1.2. Optimization Platform

Despite their practical relevance, MILP models are often difficult to construct, validate, and maintain without substantial mathematical and technical expertise. Users must translate domain knowledge into formal constraints, ensure internal consistency of model components, and iteratively debug infeasible or suboptimal formulations. To reduce this barrier, prior work has developed a collaborative web-based Optimization Platform [23, 4, 22, 20] that supports modeling and solving MILP instances through an interactive user interface.

The platform includes a visual drag-and-drop modeler, support for multiple parallel workspaces, and real-time collaboration between multiple users. Beyond usability, these features establish an iterative modeling workflow in which users can incrementally formulate, inspect, and refine optimization models while preserving shared context across collaborators. Consequently, the platform addresses both accessibility and productivity by lowering entry barriers for non-experts and reducing coordination overhead in team-based modeling scenarios.

1.3. Human-AI Collaboration

Recent advances in artificial intelligence, and in particular the emergence of transformer-based large language models (LLMs) [27], have enabled new forms of human-AI collaboration in technical workflows. Rather than functioning only as passive assistants, modern AI systems can participate in iterative problem solving, contextual interpretation, and tool-mediated task execution. In software and engineering contexts, this shift has already demonstrated that language-based interaction can reduce operational friction by allowing users to formulate complex intentions in natural language while delegating implementation-level actions to AI-supported systems.

Motivated by this development, this thesis investigates how AI-driven collaboration can be integrated into optimization modeling workflows to further improve platform usability, modeling efficiency, and support for users with heterogeneous levels of domain expertise. The central premise is that an AI agent can act not merely as a text-based helper but as an active collaborator embedded in the modeling environment, capable of proposing context-aware model modifications while remaining under explicit human supervision.

2. Challenges

Although the current optimization platform already provides a strong collaborative and user-oriented foundation, several limitations remain that constrain modeling efficiency, accessibility, and long-term usability in practice. In particular, the following challenges motivate the need for an AI-assisted collaboration layer in the proposed system:

(i) **Inefficiency in Complex Modeling Workflows:** The visual drag-and-drop interface improves usability for small to medium formulations, but can become increasingly time-intensive for large or highly structured MILP models. Repetitive modeling actions, frequent structural adjustments, and manual consistency checks introduce significant overhead. As model complexity grows, this interaction pattern can reduce development velocity and increase the risk of modeling errors that are difficult to detect without systematic support mechanisms.

(ii) **Accessibility and Learning-Curve Barriers:** The platform offers a rich set of functionalities, but this feature depth can create a substantial onboarding burden for users with limited optimization background. New users must simultaneously

understand domain concepts, interface mechanics, and modeling conventions before they can contribute effectively. Consequently, the initial learning curve remains steep, which may limit adoption and delay productive use in educational and applied settings.

(iii) **Limited Real-Time Guidance for Individual Users:** While collaborative sessions with multiple human participants can provide peer support, users working independently often lack immediate assistance during model construction, debugging, or refinement. In such scenarios, unresolved uncertainty can interrupt workflow continuity and lead to trial-and-error interactions that are both slow and error-prone. A context-aware assistant capable of delivering on-demand guidance and actionable model suggestions can therefore improve both user confidence and overall modeling quality.

3. Goal

The objective of this thesis is to design, implement, and analyze an AI agent that functions as a first-class collaborator within an existing collaborative optimization platform. In contrast to conventional assistant paradigms that remain external to the modeling process, the proposed agent is embedded directly into the shared workspace and participates through the same collaboration interface used by human users. Operationally, the agent receives natural-language instructions, interprets them with respect to the current model state, and generates candidate modifications to model structures, constraints, and related workspace artifacts.

A central design principle of this work is human-centered control and accountable collaboration. Consequently, agent-generated modifications are treated as provisional artifacts rather than immediately committed updates. Users can inspect proposed changes, approve or reject them, and provide iterative feedback that is incorporated in subsequent generation cycles. This interaction model establishes a human-in-the-loop workflow that combines automation with user supervision and preserves human authority over final model decisions.

To improve reliability, the architecture incorporates a validation stage between generation and commitment. This stage performs syntactic and structural checks on generated model artifacts and filters invalid, inconsistent, or hallucinated outputs before they affect the shared workspace. As a result, the system aims to maintain modeling correctness while still benefiting from rapid AI-assisted iteration.

At a research level, this thesis seeks to answer the following research questions:

- Can an integrated, stateful, and supervised agent architecture reduce modeling effort for routine and repetitive tasks?
- Can such an architecture lower entry barriers for users with limited optimization expertise?
- Can it improve the efficiency and quality of collaborative MILP model development?

The overarching goal is therefore not only to introduce AI-generated assistance, but to operationalize it as a reliable collaboration mechanism within a production-oriented optimization environment.

4. Solution Outline

This section provides a structured overview of the proposed system architecture and its interaction workflow. At a high level, the solution combines four tightly coupled elements: (i) a collaborative modeling workspace, (ii) an LLM-based agent that interprets user intent, (iii) an iterative feedback mechanism for human supervision, and (iv) a validation layer that enforces model-level correctness constraints before commitment. Together, these components form a stateful human-AI collaboration loop designed for optimization modeling tasks.

4.1. AI-Agent Collaborator

The AI agent is integrated into the Optimization Modeler as a workspace participant rather than an external utility. After a user creates or opens a model, the agent can be invited through the existing collaboration infrastructure and becomes part of the shared session context. This design choice preserves architectural consistency with human collaboration while enabling the agent to access the same model state, interaction history, and relevant workspace metadata required for context-aware assistance.

Functionally, the agent acts as an execution-capable collaborator: it receives user instructions, maps them to model-level operations, and generates candidate changes in a form that can be inspected within the same collaborative environment. By embedding the agent directly into the session, the system avoids context fragmentation between separate tools and supports a unified modeling workflow.

4.2. Iterative Feedback

The collaboration workflow follows an iterative proposal-and-review pattern. Users express modeling intentions in natural language, after which the agent generates a candidate update based on the current workspace state. Instead of committing changes immediately, the system exposes the proposal for explicit user evaluation. Users can approve the proposal, reject it, or provide corrective feedback that is incorporated into subsequent iterations.

This interaction model serves two goals simultaneously: it leverages AI assistance to accelerate routine modeling operations, and it preserves human control over semantically critical decisions. Iterative refinement also enables progressive convergence toward higher-quality model formulations, especially in cases where the initial prompt is ambiguous or under-specified.

4.3. Safety and Validation

A dedicated validation stage is executed between agent generation and final model commitment. This stage performs syntactic and structural checks to ensure that generated artifacts are compatible with platform constraints and do not violate core modeling assumptions. Outputs that fail validation are rejected or returned for further refinement, preventing malformed or hallucinated updates from being applied to the shared workspace.

From a systems perspective, this validation layer is essential for reliable AI integration in optimization environments: it acts as a safeguard against invalid transformations while maintaining workflow continuity for valid proposals. Consequently, safety is treated as an operational property of the collaboration pipeline rather than a post hoc correction step.

Chapter II.

Related Work

1. LLMs and Mathematical Modeling

1.1. End-to-End Modeler and Solver Archirecture

Extensive research has been conducted at the intersection of Large Language Models (LLMs) and mathematical optimization. A prominent example is **OptLLM** [31], a framework designed to bridge natural language descriptions and specialized solvers. OptLLM utilizes a modular architecture consisting of three primary components: the *Interaction Refinement Module*, which processes user queries and handles multi-round clarifications; the *Converter Module*, which includes a formulator to translate text into mathematical structures and a coder to generate executable code; and finally, the *Responser Module*, which executes the model via external solvers and interprets the results for the user. This architecture demonstrates the feasibility of integrating disparate modules to transform an LLM into a comprehensive end-to-end optimization system.

1.2. Natural Langauge to Formal Specification

Another significant contribution is **LLMOPT** [13], a learning-based framework that focuses on boosting the generalization of LLMs across various optimization types. LLMOPT introduces a universal *Five-Element Formulation* that reduces complex problem descriptions into five core components: sets, parameters, variables, objectives, and constraints. This structured representation allows the system to generate consistent solver code across diverse domains, including linear, nonlinear, and combinatorial optimization.

1.3. LLM Benchmark for Optimization Modeling

While the aforementioned frameworks focus on generation, **Mamo** [10] provides a specialized benchmark to evaluate these modeling capabilities. Mamo consists of 1,209 meticulously curated questions spanning Ordinary Differential Equations (ODEs), Linear Programming (LP), and Mixed-Integer Linear Programming (MILP). Unlike traditional result-oriented metrics, Mamo utilizes a "model generation and numerical validation" approach. An LLM is tasked with formulating a model from the dataset, which is then

executed by a solver; the resulting numerical answer is compared against the provided ground truth. This benchmark is particularly valuable for the evaluation phase of this thesis, as it provides a standardized methodology for measuring the accuracy of AI-generated mathematical models.

2. AI Code Generation

2.1. Current State

Since the introduction of the Transformer architecture [27], there has been a paradigm shift in the field of automated software engineering. While Transformers were initially celebrated for their success in Natural Language Processing (NLP), their ability to model long-range dependencies and complex structural patterns has led to significant advancements in Large Language Models (LLMs) for code generation.

Extensive research has explored the transition from general-purpose text generation to domain-specific code synthesis. As detailed in the comprehensive survey by Jiang et al. [14], modern frameworks leverage massive datasets of source code to perform tasks such as code completion, bug detection, and natural-language-to-code translation. The survey categorizes state-of-the-art models—including GPT-4, CodeLlama, and StarCoder—and examines the underlying training objectives, such as "Fill-in-the-Middle" (FIM) and instruction tuning, which are essential for understanding the non-linear structure of programming languages.

2.2. Coding Agents

Recent advancements in AI-powered coding assistants—such as GitHub Copilot [24], Google Gemini CLI [8], and Anthropic's Claude Code [2]—demonstrate how generative AI significantly enhances software development workflows. These coding agents operate as collaborative tools that integrate directly into development environments, where they utilize Large Language Models (LLMs) fine-tuned on vast code and documentation corpora [30]. Typically, users write natural language prompts or partial code snippets; the agent responds by generating, refactoring, or completing code, suggesting best practices, and occasionally identifying bugs.

Most modern coding agents rely on transformer-based architectures pre-trained on publicly available open-source repositories, and are further tuned for developer intent and task relevance through techniques such as Reinforcement Learning from Human Feedback (RLHF) and tool integration APIs [30]. For example, GitHub Copilot, powered by OpenAI's Codex model, integrates seamlessly into IDEs and proactively suggests entire code blocks in real-time [24]. Google's Gemini CLI and Anthropic's Claude Code Agent similarly offer advanced contextual understanding, enabling cross-file reasoning and multi-modal support such as code explanation, completion, or test synthesis.

Despite their power, these tools exhibit key limitations when applied to mathematical optimization modeling [7]. Primarily, their training prioritizes general-purpose software

engineering tasks—such as web development, scripting, or library usage—rather than symbolic reasoning or formal specification needed for optimization problems. As a result, these agents may struggle with generating accurate and semantically valid mathematical models, including algebraic constraints, variable declarations, and objective functions, particularly when such constructs must strictly adhere to domain-specific formats. Furthermore, their validation mechanisms are largely confined to code execution or basic unit tests, whereas optimization modeling requires both syntactic correctness and adherence to deeper mathematical requirements.

Thus, while coding agents have greatly improved code generation and developer productivity in mainstream programming, their architecture and underlying datasets are not inherently optimized for the highly structured, domain-specific tasks characteristic of mathematical optimization modeling.

2.3. LLM Benchmarking for Code Generation

As Large Language Models (LLMs) continue to evolve, the demand for more rigorous evaluation methodologies has led to the development of advanced benchmarking frameworks. One such framework is **EvalPlus** [18], which addresses the limitations of traditional code generation benchmarks.

EvalPlus builds upon established datasets, most notably *HumanEval* [5] and *MBPP*. While these original benchmarks provided a foundational method for measuring functional correctness, their test suites are often sparse and fail to catch edge-case failures, allowing incorrect code to pass as correct. EvalPlus mitigates this by automating the synthesis of thousands of additional test cases for each problem. By significantly increasing the test-case density—often by a factor of 80x compared to the original suites—EvalPlus creates a more robust evaluation environment that exposes "false positives" in LLM-generated code.

3. Human-in-the-loop Architecture

3.1. Agentic Development Environments

Recent years have witnessed the emergence of agentic Integrated Development Environments (IDEs) that proactively embed AI agents into the user's workflow. Notable examples include **Cursor** [6], Google's **Antigravity** [3], and the Copilot extensions for Visual Studio Code [28]. These environments aim to facilitate a tight human-in-the-loop collaboration by making intelligent assistants easily available during the development process.

For instance, **Cursor** offers an AI agent that not only generates and edits code but also explains suggestions, processes natural language commands, and supports high-level refactoring through conversational interfaces directly within the code editor. Google's **Antigravity** further pushes this concept by providing contextual tool recommendations and automated fixes based on the developer's current task, often prompting the user

for confirmation or guidance before performing significant actions. Similarly, Copilot in Visual Studio Code integrates an AI agent trained on vast open-source codebases, providing real-time code completions and suggestions that the human developer can freely accept, reject, or modify as needed.

In all these platforms, the human developer remains a principal decision-maker—reviewing, editing, or vetoing AI-generated proposals. This forms a continuous feedback loop where the agent assists, but all critical actions are subject to user oversight and approval, embodying the essence of ''human-in-the-loop'' design for code authoring environments.

However, while these agentic IDEs excel in integrating AI assistance for general-purpose software development, their capabilities are typically confined to code generation, editing, and debugging. They are not inherently equipped for specialized tasks such as mathematical modeling, formal specification, or symbolic reasoning outside traditional software engineering domains. As a result, their human-in-the-loop mechanisms and agentic workflows remain primarily limited to mainstream coding activities, rather than broader, domain-specific applications like mathematical optimization modeling.

3.2. Domain-Specific Agentic Systems

Recent advancements have witnessed the emergence of domain-specific agentic systems—AI assistants designed with a focus on particular industries, products, or workflows, rather than general-purpose programming. These agents are embedded into platforms to automate, streamline, and augment specialized tasks through close integration, fine-tuned models, and continual user-driven feedback.

One notable example is **v0** [26], an AI-powered design assistant specializing in rapid user interface (UI) prototyping. V0 enables designers to describe layouts and components in natural language, then instantly generates production-grade code tailored to target frameworks like React or Vue. The system's effectiveness is a result of fine-tuning large language models on datasets encompassing real-world design systems, annotated UIs, and established coding conventions, ensuring that outputs are compatible with prevailing design and engineering standards.

Industry-leading productivity suites have also embraced agentic AI. Both the **Microsoft Copilot suite** [19] (integrated into Word, Excel, PowerPoint, Outlook, etc.) and the AI features within the **Google Workspace** [29] suite (Docs, Sheets, Slides, Gmail, and more) exemplify domain-specific assistants for business documentation and office workflows. These agents draft emails, generate meeting summaries, synthesize data into insights, automate spreadsheet analyses, and manage scheduling—all by leveraging AI models refined with proprietary enterprise data, fine-tuned user interactions, and organization-specific context. Their integration is further enhanced by enforcing strict access controls, supporting privacy-preserving computations, and continuously learning from authenticated user behavior within organizational environments.

In the academic and technical writing domain, platforms like **Overleaf** [1] have integrated targeted AI helpers that support LaTeX authors by providing syntax correction, auto-completion of citations, formatting suggestions for mathematical content, and real-time collaborative editing tips. These capabilities are achieved by specializing large

language models with academic writing data and technical document corpora, ensuring that suggestions are both technically accurate and contextually appropriate.

In addition to these deep integrations, a ubiquitous trend is the proliferation of AI helper chatbots directly on product, SaaS, and documentation pages. A representative example is the AI assistant available on the **Assistant-UI** documentation page [12], where an integrated chatbot guides users through setup processes, answers technical questions, and helps to resolve onboarding or configuration issues in real time. Such conversational agents are designed to offer immediate, context-aware assistance—addressing product queries, helping new users get started, and providing proactive troubleshooting or feature recommendations. Their models are typically trained on product documentation, frequently asked questions, and anonymized user interactions, ensuring that responses are both relevant and tailored to the specific service. Many vendors fine-tune these models to reflect evolving user needs and workflows, continually enhancing the effectiveness and relevance of automated customer support.

Collectively, these domain-specific agentic systems illustrate a shift toward AI agents that are purpose-built, data-driven, and closely aligned with the specialized tasks and environments of their target users. Through model fine-tuning, deep platform integration, and continual feedback, they offer contextually intelligent support that goes well beyond what is achievable with general-purpose language models.

3.3. Magentic-UI

Magentic-UI [21] is a human-in-the-loop (HITL) research prototype by Microsoft Research, designed to facilitate complex web-based tasks through close collaboration between the user and an AI agent. In contrast to fully autonomous agents, Magentic-UI foregrounds the user's role as an active collaborator. The system processes natural language queries from the user, interprets intent, and generates a structured, multi-step plan for task execution.

Central to Magentic-UI are its advanced interaction mechanisms: *co-planning* and *action guards*. Through co-planning, the agent transparently presents its proposed sequence of actions, allowing the user to review, modify, approve, or halt steps before anything is carried out. Action guards further require explicit human approval for any pivotal or potentially sensitive operations, such as sharing confidential data or triggering transactions. This layered approach ensures user oversight at key decision points, fostering transparency and safety throughout the process. By embedding human review and iterative feedback into the agent's workflow, Magentic-UI exemplifies contemporary best practices in human-in-the-loop AI architectures—especially for web-based, agentic environments—highlighting the synergy of AI assistance and maintained human authority.

Chapter III.

Concept

1. Overview

The proposed concept models the AI agent as a *first-class collaborator* in the existing Optimization Platform instead of treating it as an external helper. This design decision follows the same collaboration paradigm already established for human users in prior platform work [23, 4, 22, 20]: the agent joins a shared workspace, observes the current model state, proposes modifications, and interacts with users through the same operational context.

At a system level, the concept consists of five tightly integrated layers:

(i) **Optimization workspace layer**, where users and the agent operate on a shared model state.

(ii) **LLM reasoning layer**, which translates natural-language instructions into domain-specific model edits.

(iii) **Interaction layer**, which provides a persistent chat interface inside the modeler.

(iv) **Execution layer**, which applies generated edits to the web modeler through controlled browser automation.

(v) **Control and validation layer**, which enforces human oversight and structural correctness before final acceptance.

This chapter discusses each conceptual pillar, compares relevant alternatives, and justifies the selected design with respect to the implementation constraints of the platform.

2. Optimization Modeling

The Optimization Platform is centered around a visual modeler for mixed-integer linear programming (MILP), where models are represented as structured markup and rendered as editable workspace content. In this environment, modeling is inherently stateful: every change affects subsequent editing, collaboration, validation, and solving steps.

Multi-user collaboration is realized through a shared workspace session with role-based access links (write/edit versus read-only), following the collaboration model established

in earlier platform iterations [23, 4]. Once users join the same workspace, all participants operate on a common model state instead of maintaining independent local copies. Conceptually, each edit is treated as an event against this shared state and is propagated to connected collaborators through the session channel, so that structural changes in the modeler are reflected across participants with minimal delay.

Change propagation therefore follows a *single-source-of-truth* pattern: updates are applied to the active workspace representation, broadcast to collaborators, and rendered in their current modeler context. This reduces merge conflicts typical of file-based workflows and enables synchronous co-modeling. At the same time, role separation remains important: users with write permissions can produce state transitions, while read-only participants observe the same evolving model without mutating it.

From a conceptual perspective, this has two direct implications for AI integration. First, the agent must operate on the *current workspace state* rather than on isolated prompts. Second, model updates must preserve structural consistency with the platform-specific schema used by the modeler. Consequently, the agent is not designed as a generic text completion system; it is designed as a context-aware transformation component that produces valid, executable modeling artifacts.

The proposed concept therefore adopts an in-place collaboration strategy: the AI agent is invited into the same workspace channel as human collaborators and acts on the same editable model representation. This avoids synchronization issues between separate tools, preserves the platform's existing collaboration semantics, and keeps the user in a familiar workflow.

3. Large Language Models (LLMs)

Large Language Models (LLMs) provide the semantic interface between human intent and formal model edits. In this thesis, the LLM layer is responsible for mapping natural-language requests to platform-specific markup while respecting optimization semantics and syntactic constraints [27, 7]. Unlike standard code completion tasks, this mapping requires both domain-aware reasoning and strict output-shape control.

At a technical level, contemporary LLMs are predominantly transformer-based sequence models that predict the next token conditioned on prior context [27]. During pretraining, models ingest large-scale text and code corpora and learn statistical regularities of language, structure, and reasoning patterns. After pretraining, they are typically adapted through instruction tuning and preference alignment so that they better follow user intent, produce more useful responses, and reduce unsafe behavior. In practical use, user prompts are converted into token sequences, processed through stacked self-attention and feed-forward layers, and decoded into output tokens that form the final response.

The current LLM landscape is characterized by rapid iteration along three dimensions: (i) *capability*, including stronger reasoning and code generation performance; (ii) *interaction modality*, including support for text, code, and increasingly multimodal inputs; and (iii) *deployment model*, spanning proprietary hosted APIs and open-weight/self-hosted alternatives [14, 30]. For engineering applications, this means model choice is

no longer purely about benchmark scores; it is also about latency, cost, controllability, context-window limits, and integration ergonomics.

In this context, an *LLM API* is the operational interface that allows external systems to call a hosted model service programmatically. Instead of running the model weights locally, the client sends structured requests (e.g., messages, system instructions, tool schemas, generation parameters) to a remote endpoint and receives generated outputs, often with optional streaming. A representative example is the OpenAI API, where applications send chat-completion style requests and receive token-generated responses that can be consumed in real time. Conceptually, APIs decouple model consumption from model hosting: the application focuses on orchestration logic, while model serving, scaling, and infrastructure maintenance remain with the provider.

For this thesis, that API-based design is particularly beneficial. It enables fast experimentation across models, simplifies integration with orchestration frameworks, and supports tool-calling patterns required for stateful, human-in-the-loop workflows. At the same time, API usage introduces practical constraints (cost, quota, rate limits, external dependency), which must be handled at the system-design level.

3.1. Alternatives

Several alternatives are plausible for this layer:

- **Single-provider LLM integration:** Directly binding the system to one model vendor (e.g., only OpenAI or only Anthropic) simplifies initial setup but increases vendor lock-in and limits model experimentation.
- **Local/self-hosted models only:** This improves deployment control and potentially data governance, but may reduce generation quality for complex optimization prompts unless substantial infrastructure and tuning effort are invested.
- **Model-provider abstraction :** A dedicated routing/proxy layer under full application control can unify multiple vendor APIs behind one interface, but introduces an additional service to operate, monitor, and secure.

3.2. Choice

The implemented concept uses **OpenRouter** as the model-provider abstraction layer. OpenRouter is a hosted routing service that exposes an OpenAI-compatible API surface while allowing requests to be dispatched to different underlying model providers through a unified interface. In practice, the client sends a standard chat-completion style request to OpenRouter together with a model identifier; OpenRouter then forwards the request to the selected backend model and returns the generated response in a compatible format.

Relative to alternatives such as a custom provider gateway or a LiteLLM-based self-managed abstraction, this choice favors lower operational overhead in the thesis context: multi-model routing is available immediately without introducing an additional in-house proxy service that must be deployed and maintained. This choice is preferable for four reasons:

(i) **Model flexibility:** The architecture can switch or compare model backends with minimal pipeline changes.

(ii) **Stateful control flow:** Graph-based nodes and transitions naturally support iterative generation, tool calls, and interruption points for user feedback.

(iii) **Domain-constrained prompting:** A persistent system prompt can encode strict grammar and validation rules for the optimization markup, improving output consistency for modeling tasks.

(iv) **Pragmatic integration and maintenance:** OpenAI-compatible APIs reduce integration friction with existing libraries, while avoiding the extra reliability and observability burden of running a separate gateway component.

In short, the selected design optimizes for adaptability and controllability, which are more important for this thesis than maximizing benchmark performance on generic coding tasks.

4. Web Automation

The AI agent must not only generate edits but also apply and retrieve workspace content in a live web session. This requires a browser automation layer capable of authenticated login, workspace navigation, DOM extraction, and deterministic DOM updates.

Conceptually, web automation is the programmatic control of a browser instance to reproduce user-level interactions with a web application. Instead of invoking internal functions directly, an automation script executes actions at the interface level (e.g., open page, fill form fields, click controls, read DOM state), which makes the behavior closer to real end-user operation. This is particularly relevant when key platform functionality is embedded in client-side logic and collaborative UI workflows rather than being fully exposed through stable backend APIs.

Operationally, a web automation runtime launches a browser process, opens a page context, and then performs a sequence of scripted actions. After each action, the runtime can inspect page state through selectors and JavaScript evaluation, enabling closed-loop execution: observe current state, execute next action, and verify effect. In this thesis, that mechanism is used not only for one-time setup (authentication and workspace joining), but also as a continuous execution environment where the agent can read the current model representation and apply validated updates back to the modeler DOM.

From an architectural perspective, this browser session functions as the agent's *environment interface*. The LLM decides *what* model transformation should be performed, while the automation layer is responsible for *how* that transformation is enacted in the live collaborative workspace. This separation improves modularity: reasoning remains model- and prompt-centric, whereas execution remains UI- and state-centric.

4.1. Alternatives

Common alternatives include:

- **Direct internal API integration:** If all required model operations are exposed through stable backend APIs, browser automation can be avoided. In this project, however, the collaboration and model-editing logic is heavily UI-centric.
- **Selenium-based automation:** Mature and widely supported, but often heavier for modern JavaScript-centric workflows.
- **Playwright-based automation:** Strong multi-browser support and robust testing ergonomics, but migration would require additional adaptation effort for existing script patterns.

4.2. Choice

The implemented concept uses **Puppeteer** [25, 9]. For the current platform architecture, this is advantageous because:

(i) **Native Chromium focus:** The modeler workflow is browser-centric and JavaScript-heavy; Puppeteer provides tight control over page navigation and DOM operations in this context.

(ii) **Low conceptual overhead:** The API supports direct element evaluation and straightforward automation scripts for login, session joining, and model updates.

(iii) **Pragmatic integration speed:** A lightweight Node bridge can expose the required browser actions as HTTP endpoints consumed by the AI orchestration layer.

Given the thesis objective (functional AI collaboration in a real platform), this choice offers the best trade-off between engineering effort and operational capability.

5. Chat-User Interaction

For first-class collaboration, the user-agent channel must be embedded directly in the modeler, maintain conversational context, and support iterative refinement instead of one-shot commands. In practice, chat interaction in this system is not treated as an auxiliary feature; it is the primary coordination mechanism between user intent and agent execution. Users describe desired model changes in natural language, inspect generated responses in the same interface, and continue with clarification or corrective feedback when needed.

From a conceptual standpoint, this interaction model has three requirements. First, the chat must remain *workspace-aware*, so that responses are grounded in the current model state rather than in isolated prompts. Second, it must support *persistent multi-turn state*, because optimization edits frequently require incremental negotiation (e.g., add a variable, then constrain it, then revise objective coefficients). Third, it must support *low-friction UX patterns* (message history, typing and loading states, structured rendering, clear action affordances) so that users can focus on model logic instead of interaction mechanics.

5.1. Alternatives

Three major alternatives were considered:

- **External chat application:** Easy to prototype, but separates communication from the modeling context and introduces coordination friction.
- **Custom chat UI from scratch:** Full control over behavior and styling, but high implementation and maintenance cost.
- **Minimal command input widget:** Reduced complexity, but insufficient for long-form clarification and multi-turn collaboration.

5.2. Choice

The selected implementation embeds an in-modeler conversational UI based on **assistant-ui** with runtime integration for stateful agent communication [12, 16]. The frontend implementation combines **React**, **shadcn/ui**-style component primitives, and **Tailwind CSS** utility styling. Together, these layers provide a composable foundation where agent-specific behavior can be integrated without rebuilding standard chat infrastructure from scratch.

Concretely, React provides declarative state management for message streams and UI updates, assistant-ui contributes domain-specific interaction primitives (thread containers, runtime providers, transport abstractions), shadcn/ui-style components provide accessible and reusable UI building blocks, and Tailwind enables rapid and consistent visual composition within the existing modeler interface. This stack lets the implementation focus on collaboration logic (thread state, model context injection, approval flow) rather than reinventing baseline UI concerns such as layout scaffolding, interaction widgets, and component wiring.

This choice is better aligned with the thesis goals because it provides:

(i) **Context-preserving interaction:** Chat and model edits live in the same workspace, reducing mental and operational context switching.

(ii) **Stateful multi-turn support:** Threaded interaction enables iterative correction, clarification, and refinement of model changes.

(iii) **Reusable integration primitives:** Runtime providers, thread components, and transport abstractions reduce boilerplate and accelerate reliable integration.

(iv) **Reduced engineering duplication:** Established UI/component tooling avoids implementing common chat features from first principles, lowering implementation risk and maintenance effort.

Conceptually, this supports the transition from “AI as autocomplete” to “AI as collaborative modeling partner.”

6. Human-in-the-Loop

Human-in-the-loop (HITL) is a collaboration concept in which AI generation and human judgment are combined in a controlled decision cycle rather than delegated to fully autonomous execution [21]. In this thesis, HITL means that the agent may propose model transformations, but the user remains the final authority for accepting, rejecting, or refining those transformations.

This concept is necessary because optimization modeling has high semantic sensitivity: an output can be syntactically valid and still be mathematically inappropriate (e.g., incorrect bounds, missing constraints, wrong objective intent). For that reason, correctness cannot rely on generation alone. It must emerge from a joint process in which the agent accelerates drafting and the user validates domain intent.

Operationally, this introduces a loop-type structure: *user request* → *agent proposal* → *validation and presentation* → *user feedback/approval* → *regeneration or commit*. Implementing this reliably requires an orchestration library that can represent explicit states, conditional transitions, tool invocation, and interruption points for human feedback.

6.1. Alternatives

At the orchestration-framework level, several alternatives are possible:

- **LangChain Agents:** Fast to prototype for tool-calling scenarios, but less explicit for graph-level state transitions in complex iterative approval flows.
- **LangGraph:** Graph-based orchestration with explicit nodes, edges, conditional routing, and interruption primitives for human approval.
- **n8n:** Strong visual workflow automation and integration ecosystem, but less specialized for fine-grained in-process conversational state management in code-centric agent loops.
- **Microsoft AutoGen / Semantic Kernel style agent frameworks:** Useful for multi-agent and enterprise integration patterns, but often heavier than needed for a single-agent, workspace-centric HITL loop.
- **Custom orchestration from scratch:** Maximum flexibility, but high implementation and maintenance overhead for retry logic, state persistence, interrupt handling, and debugging.

6.2. Choice

This thesis selects **LangGraph** as the orchestration substrate for the HITL cycle [16]. LangGraph models the workflow as an explicit state graph, which directly matches the collaboration loop required by this system: generation, tool execution, user interruption, feedback integration, and conditional continuation.

This choice is preferable for five reasons:

(i) **Native loop representation:** Iterative request-feedback-regenerate cycles can be encoded as first-class graph transitions rather than ad-hoc callback logic.

(ii) **Interrupt-driven human control:** The framework supports explicit pause-and-resume points for user approval, aligning with HITL requirements.

(iii) **Deterministic control flow:** Conditional edges make progression criteria transparent and debuggable, which is critical for thesis-grade system analysis.

(iv) **Tool-calling compatibility:** External actions (e.g., browser interaction endpoints) can be integrated as graph tools without collapsing orchestration clarity.

(v) **Balanced complexity:** Compared to custom orchestration, it reduces engineering overhead while still providing stronger control semantics than purely prompt-driven pipelines.

Accordingly, HITL in this architecture is not an auxiliary safety check but the core execution paradigm: every agent output participates in a supervised loop until either user approval is reached or the proposal is discarded.

7. Backend Services

The backend concept follows a service-oriented architecture that separates orchestration, browser interaction, and modeler integration concerns. In practice, this includes a Python-based orchestration service and a Node-based browser-control bridge.

At the infrastructure level, both components are exposed as HTTP servers. An HTTP server is a process that listens for network requests on defined endpoints (routes), executes application logic, and returns structured responses (typically JSON). This is important for the present system because it creates a clean contract between components: the modeler-facing side can call backend capabilities through stable API endpoints without depending on internal implementation details.

Using HTTP-based services offers three practical advantages in this architecture. First, it provides language-agnostic interoperability, allowing Python and JavaScript services to collaborate through a common protocol. Second, it improves modularity by enabling each service (agent orchestration, browser control, validation) to evolve independently. Third, it supports deployment flexibility, since services can run on separate processes or hosts while preserving the same request interface.

7.1. Alternatives

Architecturally, at least three alternatives exist:

- **Monolithic Node backend:** Single runtime for UI integration and browser automation, but reduced access to the Python-first LLM orchestration ecosystem.
- **Monolithic Python backend:** Strong AI ecosystem fit, but less natural for tight integration with existing JavaScript-based browser automation scripts.

- **Microservice decomposition:** Clear separation of concerns, but potentially higher deployment and observability overhead.

7.2. Choice

The selected architecture uses **FastAPI + LangGraph** for agent orchestration and a **Node/Express + Puppeteer** bridge for browser-level actions. This is superior for the current project constraints because:

(i) **FastAPI + LangGraph fit the AI orchestration problem:** LangGraph has strong and mature Python support for state graphs, conditional routing, tool-calling, and interruption-based human feedback. Pairing it with FastAPI provides a lightweight, high-performance HTTP layer for exposing orchestration endpoints in a production-friendly way.

(ii) **Python is the dominant AI integration ecosystem:** Most contemporary AI/LLM tooling, orchestration libraries, and reference implementations are Python-first, which reduces integration friction and accelerates experimentation, debugging, and iteration for agent logic.

(iii) **Node.js + Puppeteer fit browser execution semantics:** Puppeteer is JavaScript-native and directly aligned with browser automation workflows. Because the target environment is a web modeler, using JavaScript for automation is natural: language and runtime conventions match the browser platform and DOM interaction model.

(iv) **JavaScript maps directly to DOM-centric operations:** DOM querying, event triggering, and in-page script evaluation are native web concepts, and JavaScript is the browser's execution language. This makes browser-side mutation and state inspection more direct than cross-language bridging patterns.

(v) **Modular evolution:** Each service can be replaced or scaled independently (e.g., model backend changes without rewriting browser control).

This separation also supports future enhancements such as stronger validation services, improved observability, and safer deployment boundaries.

Chapter IV.

Implementation

In this chapter, we move from the conceptual design to the concrete implementation of the proposed architecture. We discuss the technologies and frameworks employed, how the individual components are interconnected, and the design decisions made during implementation.

1. High-Level System Architecture

1.1. Overview

The primary access point of the Optimization Modeler platform is the **Solver-Panel**, which is implemented as an Angular application. It is responsible for core functionalities such as user authentication, model creation and editing, and multi-user collaboration. The Solver-Panel communicates with the backend service introduced in this work, which hosts the AI agent and the auxiliary services required to support the proposed architecture.

User interaction with the AI agent takes place through a chat interface embedded within the Angular application. When the user submits a prompt via this chat interface, the request is sent to the backend API service using HTTP-based communication.

1.2. Class Hierarchy

We now describe the concrete classes involved in the architecture. The **Solver-Panel** class provides the frontend interface of the Optimization Modeler. On the backend, we implement an Express.js-based service referred to as the **AI-Agent** service. This service consists of two main components. The first is the **LLM** service, which provides an abstraction layer for interacting with large language models—in our case, OpenAI's GPT-based models—through their API. The second component is the **Puppeteer-Controller**, which is responsible for controlling a server-side browser instance using the Puppeteer automation framework. This controller enables the AI agent to interact with the Optimization Modeler by programmatically accessing and manipulating the Document Object Model (DOM).

2. Solver-Panel Frontend

This section describes the user-facing component of the Optimization Platform, referred to as the *Solver-Panel*, and its integration with the AI agent service.

2.1. User Authentication

Users begin by creating an account and authenticating via the Solver-Panel's dedicated security layer. This infrastructure manages user sessions and access control for various optimization workspaces. The underlying authentication workflows and the management of user identities are implemented following the architecture established by Begaj [4].

2.2. Internal Modeler

The core of the interface is the Optimization Modeler, a vanilla JavaScript module integrated into the Angular-based Solver-Panel application. Once authenticated, users can construct Mixed-Integer Linear Programs (MILPs) using an intuitive drag-and-drop interface. This visual representation is backed by a domain-specific, XML-like markup language. Any operation performed within the graphical Document Object Model (DOM) is automatically translated into this syntax, which serves as the source of truth for the optimization model [22, 20].

2.3. Collaboration and Agent Invitation

The platform supports multi-user collaboration through a link-sharing architecture. Users can generate and share session links that allow others to join a workspace with either read-only or editing privileges [23]. To facilitate AI assistance, we extended this mechanism to include an "Invite AI Agent" functionality.

When triggered, the frontend sends an HTTP POST request to the AI agent service's `invokeai` (or `/puppet`) endpoint. This request includes the `writeUrl` (edit link) of the current session. The backend server uses this link to launch a headless browser instance that connects to the workspace, effectively joining the session as a virtual collaborator.

2.4. Chat Interface and Request Processing

To enable natural language interaction, the modeler includes a dedicated chat interface. When a user submits a query, the frontend invokes the agent's `processrequest` (or `/openai`) API endpoint. The payload for this request consists of two primary components: the current state of the optimization model (extracted in its XML format) and the user's natural language prompt. The agent service then leverages a Large Language Model to interpret the request within the context of the existing model, generating the necessary modifications which are subsequently applied back to the workspace.

3. Browser Automation Layer

3.1. Initialization

When the API endpoint `invokeAI` is called, the `puppetInit()` method of the Puppeteer controller is executed first. This method launches an instance of a headless browser (Google Chrome) on the server, navigates to the login page of the Optimization Modeler, enters the authentication credentials associated with the AI user, and completes the login process.

After the AI user has successfully authenticated, Puppeteer is used again to navigate to the `editURL` received in the response body. Visiting this URL opens the shared instance of the optimization model within the browser. Since this browser instance runs on the backend server, the shared modeler instance can now be accessed and manipulated programmatically through the backend.

3.2. DOM Manipulation

This layer is responsible for handling DOM manipulation by leveraging the capabilities provided by the Puppeteer service. After the user submits a request through the chat interface and the LLM service generates the corresponding output, the backend accesses the `innerHTML` of the modeler instance's document body and replaces the current model with the model generated by the LLM.

This operation triggers the same internal update mechanisms as those invoked when a human user manually modifies the model through the interface. As a result, the change is propagated automatically to all users connected to the same collaborative session, ensuring consistency across all active instances.

4. LLM Integration

4.1. Context

When a user submits a request through the chat interface, the `/openaiResponse` endpoint is invoked. This endpoint receives the current state of the optimization model along with the user's prompt in the request body. The backend then forwards this information to the LLM as contextual input.

Providing the current state of the model allows the LLM to understand what the user is currently working on and the progress made so far. The quality and usefulness of the LLM's output are strongly influenced by the clarity and specificity of the user prompt; consequently, more detailed prompts generally result in more accurate and helpful responses.

4.2. Prompt Engineering

Prompt engineering plays a crucial role in generating functional and reliable outputs from large language models. While users are encouraged to formulate prompts that clearly and precisely describe their requirements, system-level prompts must also be carefully designed.

The LLM must be explicitly instructed about its role and responsibilities. In our case, the model operates as part of an AI agent whose purpose is to assist users in creating and modifying optimization models. The system prompt therefore includes a clear description of this role, as well as the current model state and the user's request.

In addition, the prompt may include one or more illustrative examples that demonstrate the expected input–output behavior. This approach leverages the few-shot learning capabilities of LLMs, enabling the model to better align its responses with the desired format and semantics.

Chapter V.

Conclusion

1. Summary

This thesis has presented a novel architecture for integrating an AI agent as a "first-class collaborator" within a web-based Optimization Platform. By bridging the gap between natural language processing and visual modeling, the proposed system addresses the core challenges of inefficiency and high entry barriers in Mixed-Integer Linear Programming (MILP).

The primary technical contribution of this work is the "Phantom User" paradigm. By utilizing a headless browser automation layer (Puppeteer), the AI agent is able to join collaborative modeling sessions as a virtual participant. This approach leverages the platform's existing Operational Transformation (OT) infrastructure, ensuring that AI-driven modifications are synchronized in real-time across all connected clients without requiring invasive changes to the legacy backend.

To ensure the reliability of the system, we implemented a multi-layered safety framework. A dedicated validation layer enforces syntactic and structural correctness of the LLM-generated XML-like markup, effectively mitigating the risk of model corruption due to hallucinations. Furthermore, a stateful "Human-in-the-Loop" workflow ensures that the user remains the ultimate authority, providing a mechanism for iterative feedback and final approval of all agent-led changes.

In conclusion, this work demonstrates that AI agents can move beyond being simple "tools" to become active, collaborative participants in complex technical domains, significantly enhancing the accessibility and productivity of optimization modeling.

2. Current Status and Next Steps

While the theoretical architecture has been established, the implementation is currently in a functional prototype stage. This section outlines the current capabilities and the prioritized milestones for future development.

2.1. Implementation Status

The core communication pipeline between the *Solver-Panel* frontend and the *AI-Agent* backend service is fully operational. A dedicated chat interface has been integrated into the modeler, enabling natural language interaction. Currently, the backend service successfully utilizes Puppeteer to join collaborative sessions and the OpenAI API to translate user prompts into model updates. The agent is capable of perceiving the model state, generating modifications, and applying them directly to the Document Object Model (DOM) of the shared workspace.

2.2. Future Milestones

To reach the final vision of a robust and secure collaborative environment, the following components are currently under development:

- **Human-in-the-Loop Implementation:** The current version applies changes directly to the model. The next phase involves implementing the full iterative feedback loop (Accept/Reject/Refine) using a stateful framework like LangGraph. This will ensure that all AI-driven modifications remain tentative until they receive explicit user approval.
- **Schema-Based Validation:** Work is ongoing to define a formal grammar for the platform's XML-like syntax. This schema will form the basis of the validation layer, allowing the agent to detect and correct "hallucinations" or syntax errors before they are presented to the user.
- **UI/UX Enhancements:** To improve the collaborative experience, further UI features will be added to provide the user with clear visual cues regarding the agent's current state (e.g., "Thinking," "Validating," or "Awaiting Review").

Bibliography

[1] *AI features — overleaf.com*. https://www.overleaf.com/about/ai-features. [Accessed 26-04-2026].

[2] Anthropic. *Claude Code Agent: A Code Assistant for Claude*. 2025. URL: https://code.claude.com/docs/en/overview.

[3] *antigravity.google*. https://antigravity.google/. [Accessed 26-04-2026].

[4] Deni Begaj. *A multi-user platform for managingand solving Mixed Integer Linear Optimization problems*. Master Thesis at Saarland University. 2020.

[5] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: https://arxiv.org/abs/2107.03374.

[6] *Cursor · Agent — cursor.com*. https://cursor.com/product. [Accessed 26-04-2026].

[7] Ali Forootani. *A Survey on Mathematical Reasoning and Optimization with Large Language Models*. 2025. arXiv: 2503.17726 [cs.AI]. URL: https://arxiv.org/abs/2503.17726.

[8] Google. *Gemini CLI: A Command-Line Interface for Gemini*. 2025. URL: https://geminicli.com/.

[9] *Home — chromium.org*. https://www.chromium.org/chromium-projects/. [Accessed 16-01-2026].

[10] Xuhan Huang et al. *LLMs for Mathematical Modeling: Towards Bridging the Gap between Natural and Mathematical Languages*. 2025. arXiv: 2405.13144 [cs.AI]. URL: https://arxiv.org/abs/2405.13144.

[11] *Integer programming — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Integer%20programming&oldid=1325150256. 2026.

[12] *Introduction — assistant-ui.com*. https://www.assistant-ui.com/docs. [Accessed 26-04-2026].

[13] Caigao Jiang et al. *LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch*. 2025. arXiv: 2410.13213 [cs.AI]. URL: https://arxiv.org/abs/2410.13213.

[14] Juyong Jiang et al. "A Survey on Large Language Models for Code Generation". In: *ACM Transactions on Software Engineering and Methodology* 35.2 (Jan. 2026), pp. 1–72. ISSN: 1557-7392. DOI: 10.1145/3747588. URL: http://dx.doi.org/10.1145/3747588.

[15] Rajendra Kunwar and HP Sapkota. “An introduction to linear programming problems with some real-life applications”. In: *European Journal of Mathematics and Statistics* 3.2 (2022), pp. 21–27.

[16] *LangGraph — langchain.com*. https://www.langchain.com/langgraph. [Accessed 16-01-2026].

[17] *Linear programming — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Linear%20programming&oldid=1329325329. 2026.

[18] Jiawei Liu et al. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. 2023. arXiv: 2305.01210 [cs.SE]. URL: https://arxiv.org/abs/2305.01210.

[19] *Microsoft 365 Copilot | KI-Produktivitätswerkzeuge für die Arbeit — microsoft.com*. https://www.microsoft.com/de-de/microsoft-365-copilot. [Accessed 26-04-2026].

[20] Maksim Moor. *From No-Code to Low-Code: Augmenting Visual Modeling for Mixed-Integer Optimization*. Bachelor Thesis at Saarland University. 2024.

[21] Hussein Mozannar et al. *Magentic-UI: Towards Human-in-the-loop Agentic Systems*. 2025. arXiv: 2507.22358 [cs.AI]. URL: https://arxiv.org/abs/2507.22358.

[22] Marco Mrsic. *An interface to model and solve Mixed-Integer Linear Programs*. Bachelor Thesis at Saarland University. 2020.

[23] Marco Mrsic. *Real-Time Collaborative Optimization Platform*. Master Thesis at Saarland University. 2023.

[24] OpenAI. *Copilot: A Code Assistant for Copilot*. 2025. URL: https://github.com/features/copilot.

[25] *Puppeteer | Puppeteer — pptr.dev*. https://pptr.dev/. [Accessed 16-01-2026].

[26] *v0 by Vercel - Build Agents, Apps, and Websites with AI — v0.app*. https://v0.app/. [Accessed 26-04-2026].

[27] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[28] *Visual Studio Code - The open source AI code editor | Your home for multi-agent development — code.visualstudio.com*. https://code.visualstudio.com/. [Accessed 26-04-2026].

[29] Google Workspace. *AI Tools for Business | Google Workspace — workspace.google.com*. http://workspace.google.com/solutions/ai/. [Accessed 26-04-2026].

[30] Jian Yang et al. *From Code Foundation Models to Agents and Applications: A Comprehensive Survey and Practical Guide to Code Intelligence*. 2025. arXiv: 2511.18538 [cs.SE]. URL: https://arxiv.org/abs/2511.18538.

[31] Jihai Zhang et al. *Solving General Natural-Language-Description Optimization Problems with Large Language Models*. 2024. arXiv: 2407.07924 [math.OC]. URL: https://arxiv.org/abs/2407.07924.

Appendix A.

An Appendix